

Healthcare Insurance Capstone Project

1. Purpose

This document defines the target Redshift schema design for the Healthcare Insurance Capstone Project. It includes the cleaned source tables, output tables, and the primary key and foreign key relationships required for analytics and reporting. The schema supports the project goal of building data pipelines that analyze healthcare insurance data to improve revenue, understand customer behavior, customize offers, and calculate royalties.

2. Target Platforms

- AWS S3 for raw input storage
- PySpark / Databricks / EMR for data cleaning and transformation
- AWS Redshift for target warehouse tables

3. Source Datasets to be Cleaned

The project requires cleaning at least these datasets before loading to Redshift:

- Patients
 - Subscriber
 - Claims
 - Group_subgroup
-

4. Redshift Schema Design

Schema 1: **project_core**

This schema stores the cleaned master data.

Table 1: **patients**

Stores patient-level healthcare details.

Column Name	Data Type	Constraint	Description
-------------	-----------	------------	-------------

patient_id	INT	PRIMARY KEY	Unique patient identifier
patient_name	VARCHAR(100)	NOT NULL	Patient full name
age	INT		Patient age
gender	VARCHAR(20)		Patient gender
disease	VARCHAR(100)		Diagnosed disease
hospital_id	INT	FOREIGN KEY	Hospital reference
city	VARCHAR(100)		Patient city
admission_date	DATE		Admission date
surgery_type	VARCHAR(100)		Surgery details
insurance_type	VARCHAR(30)		Cashless/Reimbursement

Table 2: subscriber

Stores insurance subscriber and premium details.

Column Name	Data Type	Constraint	Description
subscriber_id	INT	PRIMARY KEY	Unique subscriber identifier
patient_id	INT	FOREIGN KEY	Linked patient
age	INT		Subscriber age
group_id	INT	FOREIGN KEY	Policy group id
subgroup_id	INT	FOREIGN KEY	Policy subgroup id
monthly_premium	DECIMAL(12,2)		Monthly premium amount
policy_type	VARCHAR(30)		Government/Private
subscription_date	DATE		Policy subscription date

Table 3: claims

Stores claim transaction details.

Column Name	Data Type	Constraint	Description
claim_id	INT	PRIMARY KEY	Unique claim id
patient_id	INT	FOREIGN KEY	Linked patient
subscriber_id	INT	FOREIGN KEY	Linked subscriber
disease	VARCHAR(100)		Disease for claim
hospital_id	INT	FOREIGN KEY	Linked hospital
claim_amount	DECIMAL(12,2)		Claim amount
claim_status	VARCHAR(30)		Approved/Rejected
claim_date	DATE		Date of claim
city	VARCHAR(100)		Claim city
total_charges	DECIMAL(12,2)		Total hospital charges

Table 4: group_subgroup

Stores policy group and subgroup mapping.

Column Name	Data Type	Constraint	Description
subgroup_id	INT	PRIMARY KEY	Unique subgroup id
group_id	INT	FOREIGN KEY	Linked group id
subgroup_name	VARCHAR(100)		Subgroup name
group_name	VARCHAR(100)		Group name

Table 5: policy_group

Stores parent group information.

Column Name	Data Type	Constraint	Description
group_id	INT	PRIMARY KEY	Unique group id
group_name	VARCHAR(100)	NOT NULL	Group name
group_category	VARCHAR(30)		Government/Private

Table 6: `hospital`

Stores hospital reference information.

Column Name	Data Type	Constraint	Description
hospital_id	INT	PRIMARY KEY	Unique hospital id
hospital_name	VARCHAR(150)	NOT NULL	Hospital name
city	VARCHAR(100)		Hospital city
state	VARCHAR(100)		Hospital state

5. Primary Key and Foreign Key Relationships

Primary Keys

- `patients.patient_id`
- `subscriber.subscriber_id`
- `claims.claim_id`
- `group_subgroup.subgroup_id`
- `policy_group.group_id`
- `hospital.hospital_id`

Foreign Keys

- `subscriber.patient_id` → `patients.patient_id`

- `subscriber.group_id` → `policy_group.group_id`
- `subscriber.subgroup_id` → `group_subgroup.subgroup_id`
- `claims.patient_id` → `patients.patient_id`
- `claims.subscriber_id` → `subscriber.subscriber_id`
- `claims.hospital_id` → `hospital.hospital_id`
- `patients.hospital_id` → `hospital.hospital_id`
- `group_subgroup.group_id` → `policy_group.group_id`

This structure supports the required analytics on claims, patients, hospitals, groups, subgroups, premiums, rejected claims, and government/private policy analysis described in the capstone.

6. Output Schema for Use Cases

The capstone requires a **separate Redshift table for each use case output** in a dedicated output schema.

Schema 2: `project_output`

1. `disease_max_claims`

Column Name	Data Type
disease	VARCHAR(100)
total_claims	INT

2. `subscribers_age_lt_30`

Column Name	Data Type
subscriber_id	INT
patient_id	INT
age	INT
subgroup_id	INT

3. `group_with_max_subgroups`

Column Name	Data Type
group_id	INT
group_name	VARCHAR(100)
subgroup_count	INT

4. hospital_most_patients

Column Name	Data Type
hospital_id	INT
hospital_name	VARCHAR(150)
patient_count	INT

5. most_subscribed_subgroup

Column Name	Data Type
subgroup_id	INT
subgroup_name	VARCHAR(100)
subscription_count	INT

6. rejected_claims_count

Column Name	Data Type
total_rejected_claims	INT

7. claims_by_city

Column Name	Data Type
city	VARCHAR(100)
total_claims	INT

8. policy_type_subscription_summary

Column Name	Data Type
policy_type	VARCHAR(30)
total_subscribers	INT

9. avg_monthly_premium

Column Name	Data Type
average_monthly_premium	DECIMAL(12,2)

10. most_profitable_group

Column Name	Data Type
group_id	INT
group_name	VARCHAR(100)
total_premium_amount	DECIMAL(14,2)

11. patients_below_18_cancer

Column Name	Data Type
patient_id	INT
patient_name	VARCHAR(100)
age	INT
disease	VARCHAR(100)

12. cashless_high_charge_patients

Column Name	Data Type
patient_id	INT
patient_name	VARCHAR(100)
insurance_type	VARCHAR(30)
total_charges	DECIMAL(12,2)

13. female_40_plus_knee_surgery

Column Name	Data Type
patient_id	INT
patient_name	VARCHAR(100)
age	INT
gender	VARCHAR(20)
surgery_type	VARCHAR(100)
admission_date	DATE

These output tables directly align with the required business questions listed in the capstone document.

7. SQL DDL for Core Tables

```
create schema if not exists project_core;
```

```
create table if not exists project_core.hospital (  
    hospital_id int not null,  
    hospital_name varchar(150) not null,  
    city varchar(100),  
    state varchar(100),  
    primary key (hospital_id)  
);
```

```
create table if not exists project_core.policy_group (  
    group_id int not null,  
    group_name varchar(100) not null,  
    group_category varchar(30),  
    primary key (group_id)  
);
```

```
create table if not exists project_core.group_subgroup (  
    subgroup_id int not null,  
    group_id int,  
    subgroup_name varchar(100),  
    group_name varchar(100),  
    primary key (subgroup_id),  
    foreign key (group_id) references project_core.policy_group(group_id)  
);
```



```
create table if not exists project_core.patients (  
    patient_id int not null,  
    patient_name varchar(100) not null,  
    age int,  
    gender varchar(20),  
    disease varchar(100),  
    hospital_id int,  
    city varchar(100),  
    admission_date date,  
    surgery_type varchar(100),  
    insurance_type varchar(30),  
    primary key (patient_id),  
    foreign key (hospital_id) references project_core.hospital(hospital_id)  
);
```

```
create table if not exists project_core.subscriber (  
    subscriber_id int not null,  
    patient_id int,  
    age int,  
    group_id int,  
    subgroup_id int,  
    monthly_premium decimal(12,2),  
    policy_type varchar(30),  
    subscription_date date,  
    primary key (subscriber_id),  
    foreign key (patient_id) references project_core.patients(patient_id),  
    foreign key (group_id) references project_core.policy_group(group_id),  
    foreign key (subgroup_id) references project_core.group_subgroup(subgroup_id)  
);
```

```
create table if not exists project_core.claims (  
    claim_id int not null,  
    patient_id int,  
    subscriber_id int,  
    disease varchar(100),  
    hospital_id int,  
    claim_amount decimal(12,2),  
    claim_status varchar(30),  
    claim_date date,  
    city varchar(100),  
    total_charges decimal(12,2),  
    primary key (claim_id),  
    foreign key (patient_id) references project_core.patients(patient_id),
```

```
foreign key (subscriber_id) references project_core.subscriber(subscriber_id),
foreign key (hospital_id) references project_core.hospital(hospital_id)
);
```

8. SQL DDL for Output Schema

```
create schema if not exists project_output;
```

```
create table if not exists project_output.disease_max_claims (
    disease varchar(100),
    total_claims int
);
```

```
create table if not exists project_output.subscribers_age_lt_30 (
    subscriber_id int,
    patient_id int,
    age int,
    subgroup_id int
);
```

```
create table if not exists project_output.group_with_max_subgroups (
    group_id int,
    group_name varchar(100),
    subgroup_count int
);
```

```
create table if not exists project_output.hospital_most_patients (
    hospital_id int,
    hospital_name varchar(150),
    patient_count int
);
```

```
create table if not exists project_output.most_subscribed_subgroup (
    subgroup_id int,
    subgroup_name varchar(100),
    subscription_count int
);
```

```
create table if not exists project_output.rejected_claims_count (
    total_rejected_claims int
);
```

```
create table if not exists project_output.claims_by_city (
    city varchar(100),
```

```

    total_claims int
);

create table if not exists project_output.policy_type_subscription_summary (
    policy_type varchar(30),
    total_subscribers int
);

create table if not exists project_output.avg_monthly_premium (
    average_monthly_premium decimal(12,2)
);

create table if not exists project_output.most_profitable_group (
    group_id int,
    group_name varchar(100),
    total_premium_amount decimal(14,2)
);

create table if not exists project_output.patients_below_18_cancer (
    patient_id int,
    patient_name varchar(100),
    age int,
    disease varchar(100)
);

create table if not exists project_output.cashless_high_charge_patients (
    patient_id int,
    patient_name varchar(100),
    insurance_type varchar(30),
    total_charges decimal(12,2)
);

create table if not exists project_output.female_40_plus_knee_surgery (
    patient_id int,
    patient_name varchar(100),
    age int,
    gender varchar(20),
    surgery_type varchar(100),
    admission_date date
);

```

9. Data Quality Rules

Based on the project requirements, data cleaning should include:

- check null values
 - count nulls for each column
 - replace selected null values with NA
 - detect duplicate records
 - drop duplicate records before loading to Redshift
-

10. Conclusion

This schema design supports the healthcare insurance analytics pipeline by organizing cleaned source data and output results in Redshift. It enables business reporting for all listed use cases and follows the project submission requirement for schema design, output tables, and PK/FK relationships.